

DETAILED EXPLANATION DOCUMENT

Problem 9: DAG Optimization – Redundant Expression Detection

Subject	Compiler Design / Code Optimization
Tool Used	C Programming Language (with structures & arrays)
Concept	Directed Acyclic Graph (DAG) – Common Subexpression Elimination
Features	Expression_Frequency computation, Is_Common_Subexpression flag

1. Problem Understanding

A **Directed Acyclic Graph (DAG)** is a data structure used in compiler design to represent expressions in a program. Each node in the DAG represents an operation or operand. When a compiler generates intermediate code, the same computation (e.g., $a + b$) may appear multiple times across different instructions. These repeated computations are called **Common Subexpressions (CSEs)**.

The goal of this problem is to:

- Parse a list of DAG nodes, each containing an operator and two operands.
- Build a normalized expression string (e.g., "a + b") for each node.
- Count how many times each expression appears in the DAG (Expression_Frequency).
- Set a boolean flag Is_Common_Subexpression = Yes if frequency > 1, else No.
- Display the results and a summary report to guide optimization.

Why does this matter? If the compiler detects that $a + b$ is computed 5 times, it can replace 4 of those computations with a single temporary variable, reducing the number of CPU instructions and improving runtime performance.

2. Data Model – DAGNode Structure

Each DAG node is represented using a C **struct** containing six fields:

Field	Type	Description
node_id	int	Unique identifier for the DAG node
operator_sym	char[10]	Arithmetic operator: +, -, *, /
left_operand	char[20]	Left-hand side operand (variable or temp)
right_operand	char[20]	Right-hand side operand
expression	char[50]	Computed string: "left op right"
frequency	int	How many nodes share this expression
is_common_subexpression	int	Flag: 1=Yes (CSE), 0=No

3. Solution Logic – Step by Step

Step 1 – Build Expression Strings

The function `build_expression()` constructs a canonical string "left op right" for each node using `snprintf()`. This normalizes all expressions for easy string comparison.

Step 2 – Compute Expression Frequency

The function `compute_frequency()` uses two nested loops. For each node `i`, it counts how many nodes `j` have the same expression string (using `strcmp()`). This count is stored in `nodes[i].frequency`.

Step 3 – Flag Common Subexpressions

The function `flag_common_subexpressions()` applies the threshold condition: if `frequency > 1`, then `is_common_subexpression = 1` (Yes); otherwise 0 (No). This single condition is sufficient to identify all redundant computations.

Step 4 – Display Results

The `display_table()` function prints a formatted ASCII-bordered table showing all nodes with their expression, frequency, and CSE flag. Unicode box-drawing characters create a professional table layout.

Step 5 – Summary Report

The `display_summary()` function reports unique CSE expressions with their occurrence counts, total node count, and number of optimization opportunities.

4. Key Code Snippets Explained

4a. DAGNode Structure Definition

```
typedef struct {
    int node_id;
    char operator_sym[10];
    char left_operand[20];
    char right_operand[20];
    char expression[50]; // Canonical form: "a + b"
    int frequency;       // How many times expression appears
    int is_common_subexpression; // 1 = Yes, 0 = No
} DAGNode;
```

4b. Frequency Computation ($O(n^2)$ comparison)

```
void compute_frequency(DAGNode nodes[], int count) {
    for (int i = 0; i < count; i++) {
        int freq = 0;
        for (int j = 0; j < count; j++) {
            if (strcmp(nodes[i].expression, nodes[j].expression) == 0)
                freq++;
        }
        nodes[i].frequency = freq;
    }
}
```

```
}  
}
```

4c. CSE Flag Assignment

```
void flag_common_subexpressions(DAGNode nodes[], int count) {  
    for (int i = 0; i < count; i++) {  
        // Threshold: frequency > 1 means it's a common subexpression  
        nodes[i].is_common_subexpression = (nodes[i].frequency > 1) ? 1 : 0;  
    }  
}
```

5. Justification of Threshold (frequency > 1)

The threshold of **frequency > 1** is chosen based on the definition of a Common Subexpression: an expression is "common" if and only if it appears **more than once** in the program. Any expression with frequency = 1 is unique and cannot be optimized away.

Frequency Value	Is CSE?	Reason
1	No	Expression appears only once — no redundancy
2	Yes	Appears twice — one computation can be eliminated
3+	Yes	Multiple redundancies — significant savings possible

This threshold is directly aligned with the classical **Common Subexpression Elimination (CSE)** optimization technique described in the Dragon Book (Compilers: Principles, Techniques and Tools by Aho, Lam, Sethi, Ullman). A higher threshold (e.g., > 2) would miss valid optimization opportunities.

6. Sample Output & Analysis

NodeID	Expression	Frequency	Is_Common_Subexpression
1	a + b	5	Yes ✓
2	a + b	5	Yes ✓
3	t1 * c	2	Yes ✓
4	a + b	5	Yes ✓
5	t3 - d	2	Yes ✓
6	t3 - d	2	Yes ✓
7	e * f	2	Yes ✓
8	t3 * c	1	No
9	a + b	5	Yes ✓
10	g + h	1	No
11	t5 / e	1	No
12	e * f	2	Yes ✓
13	x - y	1	No
14	a + b	5	Yes ✓
15	t1 * c	2	Yes ✓

Findings from Output:

- a + b appears 5 times (Nodes 1,2,4,9,14) — highest redundancy.
- t1 * c appears 2 times (Nodes 3,15) — can be replaced by a single temp.
- t3 - d appears 2 times (Nodes 5,6) — common subexpression.
- e * f appears 2 times (Nodes 7,12) — can be eliminated.
- t3 * c, g + h, t5 / e, x - y appear only once — NOT common subexpressions.
- 4 unique CSEs detected → 7 redundant node computations can be eliminated.

7. Algorithm Complexity

Operation	Time Complexity	Space Complexity
build_expression()	O(n)	O(n)
compute_frequency()	O(n ²)	O(1) extra
flag_common_subexpressions()	O(n)	O(1) extra
display_table()	O(n)	O(1) extra
Overall	O(n²)	O(n)

The O(n²) frequency computation is acceptable for typical basic block sizes (usually < 100 nodes). For large programs, a hash-map approach can reduce this to O(n) average time.

8. Conclusion

This C program successfully implements **DAG-based redundant expression detection** using structures, nested loops, and string comparison. The two key computed features — **Expression_Frequency** and **Is_Common_Subexpression** — provide actionable information for compiler optimization passes.

The program demonstrates a core compiler optimization principle: before generating machine code, a compiler can scan its DAG representation, identify repeated sub-trees, and reuse their results. This corresponds directly to the **Common Subexpression Elimination (CSE)** optimization pass found in production compilers like GCC and LLVM.